Enhancing LLM-Based Coding Tools through Native Integration of IDE-Derived Static Context

Yichen Li
ycli21@cse.cuhk.edu.hk
The Chinese University of Hong Kong
Department of Computer Science and Engineering
Hong Kong SAR, China

Yun Peng
ypeng@cse.cuhk.edu.hk
The Chinese University of Hong Kong
Department of Computer Science and Engineering
Hong Kong SAR, China

Yintong Huo**
ythuo@cse.cuhk.edu.hk
The Chinese University of Hong Kong
Department of Computer Science and Engineering
Hong Kong SAR, China

Michael R. Lyu
lyu@cse.cuhk.edu.hk
The Chinese University of Hong Kong
Department of Computer Science and Engineering
Hong Kong SAR, China

ABSTRACT

Large Language Models (LLMs) have achieved remarkable success in code completion, as evidenced by their essential roles in developing code assistant services such as Copilot. Being trained on in-file contexts, current LLMs are quite effective in completing code for single source files. However, it is challenging for them to conduct repository-level code completion for large software projects that require cross-file information. Existing research on LLM-based repository-level code completion identifies and integrates cross-file contexts, but it suffers from low accuracy and limited context length of LLMs. In this paper, we argue that Integrated Development Environments (IDEs) can provide direct, accurate and real-time cross-file information for repository-level code completion. We propose IDECoder, a practical framework that leverages IDE native static contexts for cross-context construction and diagnosis results for self-refinement. IDECoder utilizes the rich cross-context information available in IDEs to enhance the capabilities of LLMs of repository-level code completion. We conducted preliminary experiments to validate the performance of IDECoder and observed that this synergy represents a promising trend for future exploration.

CCS CONCEPTS

• **Software and its engineering → Automatic programming**.

KEYWORDS

Large language model, code generation

ACM Reference Format:
Yichen Li, Yun Peng, Yintong Huo, and Michael R. Lyu. 2024. Enhancing LLM-Based Coding Tools through Native Integration of IDE-Derived Static Context. In *2024 International Workshop on Large Language Models for Code (LLM4Code '24), April 20, 2024, Lisbon, Portugal*. ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3643795.3648392

** indicateds the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
LLM4Code '24, April 20, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0579-3/24/04…$15.00
https://doi.org/10.1145/3643795.3648392

1 INTRODUCTION

Inspired by the great success of large language models (LLMs) on natural language processing (NLP), code LLMs, including CodeX [19] and StarCoder [14], are proposed and obtain promising performance in code intelligence tasks such as code completion. The code LLMs broadly advance the development of code assistant services like Copilot [10] and CodeWhisperer [2]. By providing real-time, context-based code suggestions, code assistant services significantly improve developer productivity.

Similar to general LLMs, code LLMs are trained on in-file contexts [5, 14, 19] and are not aware of software project architectures. Without cross-file information such as imported methods and external classes, they struggle to understand the entire architecture of a software project and obtain significantly low performance in repository-level (repo-level) code completion [6]. Fig. 1 presents a failed case of StarCoder [14] in repo-level code completion. Even with a large knowledge base obtained in the pre-training stage, Code LLM fails to complete the logging statement since the in-file context does not provide sufficient type information of variable *Service*, which is defined in another file and may not appear in the training data. Therefore, it is essential to include cross-file information, which cannot be obtained in the internal knowledge base, for code LLMs in repo-level code completion.

To include cross-file context information, researchers have proposed various repo-level code generation frameworks [1, 3, 7, 24, 28]. Most adopt two major steps: identifying the appropriate cross-file context and logically integrating cross-file contexts. To identify cross-file contexts, they employ techniques such as identifier name matching [7] and semantics retrieval [9, 28]. For fusing cross-file contexts, they utilize methods like all-import [1] and encoding-based fusion [24]. These approaches can supplement the in-file contexts with external knowledge. However, they still suffer from unsatisfactory performance due to various factors, such as inaccurate context identification caused by complicated language features and limited context length of LLMs.

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:37:03 UTC from IEEE Xplore. Restrictio

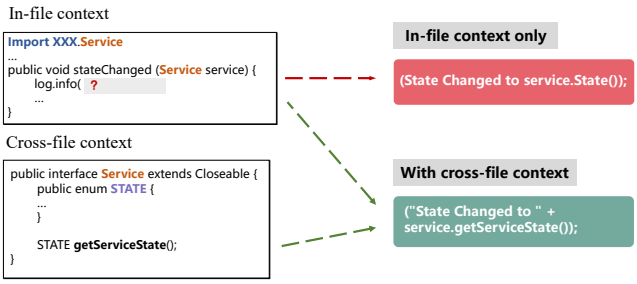

Figure 1: An example of the completed code from StarCoder. StarCoder fails to complete a Java program correctly as the in-file context does not provide sufficient type information of *Service*. To get the service state, the model needs to know the member function *getServiceState* of *Service*.

Modern Integrated Development Environments (IDEs) [12, 16] natively offering robust code navigation, suggestion and static diagnosis capabilities, are widely used by developers in software development. IDEs can manage the entire project structure and provide practical cross-file context information, such as class hierarchies, function signatures, and variable types in real time. Despite the impressive performance of current LLM-powered coding tools, they do not make use of the rich context information available within IDEs and thus struggle to handle repo-level code completion.

In this work, we argue that LLMs and LLM-powered coding tools should address code completion tasks by jointly considering in-file and cross-file contexts, leveraging the native static information and static diagnosis result (warnings, errors) directly provided by Integrated Development Environments (IDEs) [25] during development. We propose IDECoder, a novel and practical LLM-based code completion framework powered by IDE native static contexts. Our vision is to integrate native static information as static contexts and diagnosis results as feedback for self-refinement. This framework allows LLMs to utilize the wealth of cross-file information in IDEs, ultimately enhancing their capabilities to complete code that is aware of the overall project structure and dependencies. We conduct preliminary experiments of IDECoder which demonstrate the promise of IDE native code LLMs as a promising and practical direction of exploration. By integrating static contexts and diagnosis results, our framework paves the way for more sophisticated and context-aware code completion, ultimately benefiting developers in their software development.

2 REPO-LEVEL CODE COMPLETION: CHALLENGES

Repo-level code completion aims to complete code using the cross-file broader contexts. However, it faces difficulties in accessing useful and relevant information dispersed across files, such as imported classes and their member functions from different modules, global variables, and external classes with unknown type details. This process involves two primary steps: cross-file context identification and context fusion. In the following subsections, we introduce the challenges for the two steps.

2.1 Identification of Cross-file Contexts

When working with a code snippet and associated files in a repository, it is crucial to identify the most relevant and useful cross-file contexts, such as member functions of utilized variables, for code-based LLMs. This ensures accurate reference and prevents confusion for LLMs. The primary challenges in identifying cross-file contexts are maintaining *Accuracy* and ensuring *Relevance*.

Accuracy. Current retrieval-based methods [7, 15, 28] of identifying related references based on identity names and semantic similarity can hardly handle complicated program language features, such as inheritance, polymorphism, and complex namespaces, and thus leading to wrong identifications. Accurately identifying the cross-file contexts necessitates a more refined approach considering the specific language features. In contrast to Retrieval-Augmented Generation (RAG) [13] tasks based on semantic similarity, fuzzy matching is generally not applicable for code-related tasks under the soundness requirement. Therefore, there is a need for developing static analysis-enhanced retrieval approaches.

Relevance. Accurate reference and definition identification are not enough for building cross-file contexts. It is also crucial to identify relevant contexts that reflect the developer's intentions, such as code comments [7]. This helps LLMs understand the relationships across the repository, such as the expected method invocation sequences, the functional role of the current file within the entire project, and the module dependencies. Existing approaches [8, 9] provide the code LLMs with similar code snippets via in-context learning to teach code-based LLMs how to program in the current case. However, LLMs do not actually understand the developer's intention in the current project and the role of the current source file since examples come from other software projects and are not relevant to the current project.

2.2 Fusion of Cross-file Contexts

Given the collected cross-file contexts, it is essential to organize them in a way that code-based LLMs can better understand. Simply concatenating in-file and cross-file contexts is suboptimal for two reasons. First, it is impractical to include all source codes of identified import classes, due to the limited context length of LLMs. Second, overlong contexts can also hurt the performance of current code-based LLMs [7] and lead to excessive costs [20]. Current encoding-based methods [7, 24] simply fuse these contexts together, but they still need LLMs to be tuned for the new data format. Besides, approaches that rely solely on in-context learning [9] by providing input and output examples may exacerbate the drawbacks of similarity-based retrieval methods.

Furthermore, different elements in the contexts should have different importance. For example, local dependencies in the current projects are more important than popular third-party packages (i.e., Numpy [18]), as the later ones may already exist in the internal knowledge base of LLMs.

3 METHODOLOGY

The IDECoder aims to integrate static information as static contexts and diagnosis results in LLM-based code completion. As illustrate

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:37:03 UTC from IEEE Xplore. Restrictio

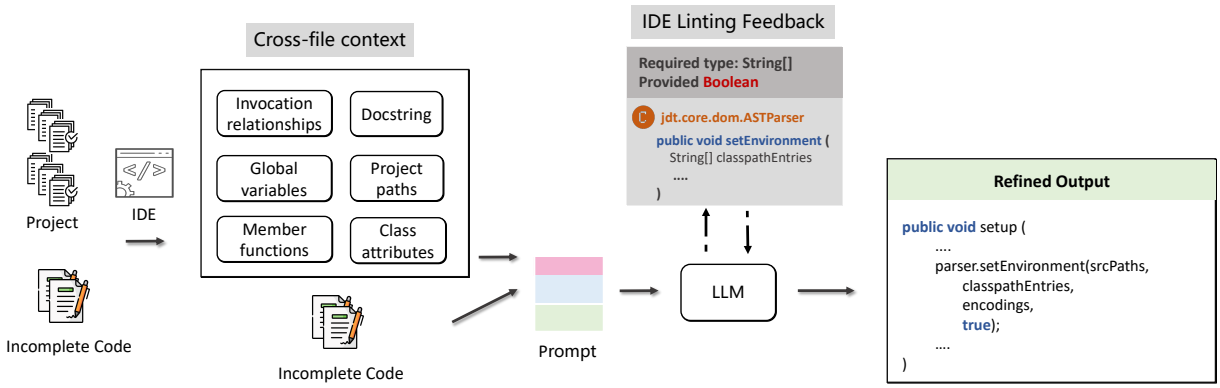

Figure 2: The overall framework of IDECoder.

in Fig. 2, IDECoder takes the target incomplete code and its corresponding project as input in IDE. By leveraging the analysis capabilities of IDEs, IDECoder accurately identifies cross-file contexts and extracts relevant information for cross-file context identification. Next, IDECoder models and organizes the identified cross-file contexts as inputs for the LLMs. It employs a chain-of-thought [21] methodology to model this information sequentially, enabling the LLM to generate more contextually relevant code completions. Finally, IDECoder refines the generated code using the diagnostic output from the IDE's linting service, which ensures the quality and correctness of the generated code.

To effectively integrate cross-file context information into the LLM-based code completion process, our methodology focuses on three key phases: *cross-file context identification*, *cross-file context fusion*, and *linting-based code refinement*.

3.1 Cross-file Context Identification

To address the challenges of cross-file context identification, we take advantage of the native capabilities of IDEs to pinpoint above mentioned cross-file contexts.

In terms of *Accuracy*, IDEs provide various features such as abstract syntax tree (AST) construction, symbol table [22] creation, reference indexing, and code element localization. By utilizing these features, IDEs can parse code, identify code elements, references and further relationships, and look up class attributes. With language-specific static analysis, IDEs can accurately identify cross-file contexts. This allows IDECoder to overcome the limitations of current retrieval-based methods [7, 27, 28] by considering language-specific features like inheritance, polymorphism, and complex namespaces.

For *Relevance*, IDEs can assist in identifying the most pertinent cross-file contexts that reflect the developers' intentions, such as docstrings of imported methods, method invocation relationships, method functionalities, and module dependencies. By understanding the relationships of different code elements within the repository, IDEs can provide valuable contextual information to LLMs, enabling them to generate code that aligns with the developers' intentions and project structure.

3.2 Cross-file Context Fusion

Given the identified cross-file contexts, IDECoder then organizes them as inputs for LLMs. Unlike previous methods such as encoding [7, 24], all import [1], reasoning [3] that use the complete code of methods in cross-file contexts, IDECoder utilizes docstrings, as well as method and class signatures with detailed type information. Docstrings indicate the developers' intentions of building the methods, while method and class signatures reflect the functionalities. Using them instead of the entire code could largely reduce the context length without losing valuable information.

To collect docstrings and method and class signatures, IDECoder first analyzes the imported packages and modules in the repository to differentiate between widely used third-party libraries and user-defined classes and methods. IDECoder incorporates different strategies for handling third-party libraries and user-defined classes and methods. For **third-party** libraries, such as NumPy [18] in Python, IDECoder briefly includes version information of the package to ensure API as the code and usage patterns of such libraries may already exist in the knowledge base of LLMs obtained in the pre-training stage [11]. For **user-defined** classes and methods, IDECoder extracts their essential properties and relationships. It then collects docstrings and method and class signatures with detailed type information acquired from class and method definitions. By leveraging such information, IDECoder can create a concise and informative representation of the cross-file context, enabling LLMs to make accurate and contextually relevant code completions.

Given the collected docstrings and method and class signatures, IDECoder does not directly list them as the inputs. Instead, it employs a chain-of-thought methodology [21] to model information, sequentially illustrating each piece of information. This approach introduces the acquired context in a top-down manner, ranging from the functional role to specific type details. This mechanism allows IDECoder to consider the sequence of code elements, their relationships, and the developers' intentions when generating code completions. By incorporating this information, IDECoder can generate more contextually relevant and coherent code snippets that adhere to the project's structure and design principles. Once the chain-of-thought prompts are constructed, they are sent to the LLMs for initial results.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:37:03 UTC from IEEE Xplore. Restrictio

3.3 Linting-based Code Refinement

Given the completed code generated by LLMs, IDECoder further conducts linting-based code refinement to ensure the quality and correctness of the generated code.

Current IDEs utilize a range of as-you-type static code inspection tools, such as Pylance [16] and ESLint [12], to identify and rectify incorrect code within a project before compilation. Once IDECoder receives completion results from LLMs, the IDE's as-you-type linting service is activated to detect potential warnings, type errors, and method usage issues.

IDECoder employs a two-step process to ensure the quality of completed code. First, it utilizes the diagnostic output from the linting service to identify issues [25] in the generated code. Then, it resends the identified issues and their previous context to the LLMs for correction and refinement. This process enhances the generated code's quality and allows IDECoder to self-improve and rectify variable usage, ensuring the generated code's syntactical correctness. In the case of unimported used methods in completed code, the import management process is optimized to efficiently maintain import packages with the suggestions of linting, overcoming the limitations of plain text style completion.

After refining the code based on the linting feedback, IDECoder-presents the corrected code to the user. Linting-based code refinement offers a more lightweight, real-time, and unobtrusive user experience compared to conversational program repair [26] using execution feedback, as it does not necessitate program execution.

4 PRELIMINARY EVALUATION

4.1 Preliminary Implementation

We access the static information by hooking into Pylance [16] as the **proof-of-concept** to showcase the effectiveness of IDECoder in this vision paper. Pylance is an extension that works alongside Python in Visual Studio Code [17] to provide Python language services for programmers. All contexts for prompt construction come from hooking into events of the Pylance plugin. For some strongly-coupled information that cannot be hooked, we **manually** collect and send them to LLMs in our experiments.

We set GPT-3.5 (the fixed version, *gpt-3.5-turbo-0301*) as the backbone model. Besides, We adjusted the *temperature* to 0 to ensure the model consistent generation, thereby ensuring reproducibility.

4.2 Evaluation Setup

Datasets. Owing to the lack of repo-level code completion datasets, we emulate the data collection pipeline from the code completion dataset that has cross-file contextual information `CROSSCODEEVAL` [6]. Other cross-file datasets [6, 7] are not applicable because they do not allow customized prompts as inputs for by offering the fixed cross-file prompts. We randomly sample 10 Python repositories from `CROSSCODEEVAL` to conduct the function body completion task [28] (functions less than 15 lines) and use the GPT-3.5 backbone model, as it can comprehend complex prompt strategies.

Metrics. In line with previous works [4, 9, 23], we evaluate the performance of code completion with three metrics including Exact Match (EM), CodeBLEU (CB) [23], and Syntax Match (SM). EM evaluates the extent to which the model-generated code is identical to the target code. CB calculates the similarity of code snippets by considering syntax and semantic information, such as data flow and Abstract Syntax Tree (AST). SM quantifies the match of subtrees within the code.

4.3 Evaluation Results

As a preliminary step, we implement three methods as baselines for comparison: an in-file completion method, an all-import method [1], and a basic RAG [28] method following the previous work [28].

Table 1: Performance comparison on the function body completion dataset. Numbers are shown in percentage (%).

Metrics	Exact Match	CodeBLEU	Syntax Match
In-file	7.37	27.84	44.17
All-import	6.71	30.53	46.24
RAG	9.77	31.65	48.92
IDECoder	**10.46**	**34.16**	**50.73**

Based on the evaluation results presented in Table. 1, IDECoder consistently outperforms all baseline methods in code completion tasks, highlighting its effectiveness. The performance of the other frameworks in these tasks underlines the existing limitations in cross-file code completion. *Note that* the current implementation of IDECoder is constrained by the closed-source Pylance plugin, which restricts full access to the aforementioned contexts. In future work, if IDE vendors develop customizable native coding tools that allow for more extensive access to cross-file contexts, the performance of the IDECoder framework could potentially surpass its current capabilities.

5 DISCUSSION AND FUTURE PLANS

Code LLMs are becoming increasingly influential and gradually integrating into the daily lives of developers, benefits millions of users. LLMs natively integrated with Integrated Development Environments (IDEs) are expected to be even more powerful and natural. The strong capabilities of IDEs in retrieving information and profiling project structures largely enhance current large-scale code models. Our preliminary experiment (seen in Sec. 4) results support this claim, demonstrating a promising research direction of learning cross-file contexts.

In the future, we plan to implement a more mature version of IDECoder by developing the IDE plugin, which can support user-defined backbone LLMs. Additionally, we will extend IDECoder to support a broader range of code-related tasks. As the core idea of extending cross-file contexts from IDE-provided static information for code intelligence tasks has shown success in this paper, it can be generalized to other code-related tasks, such as repo-level automated program repair (APR) and IDE-assisted automated debugging.

6 ACKNOWLEDGEMENT

The work described in this paper was supported by the Research Grants Council of the Hong Kong Special Administrative Region,

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:37:03 UTC from IEEE Xplore. Restrictio

China (No. CUHK 14206921 of the General Research Fund). We thank all reviewers for their valuable comments.

REFERENCES

[1] DeepSeek AI. 2023. DeepSeek Coder: Let the Code Write Itself. https://github.com/deepseek-ai/DeepSeek-Coder.

[2] Amazon. 2023. CodeWhisperer. https://aws.amazon.com/cn/codewhisperer/

[3] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B Ashok, Shashank Shet, et al. 2023. CodePlan: Repository-level Coding using LLMs and Planning. *arXiv preprint arXiv:2309.12499* (2023).

[4] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T Devanbu, and Baishakhi Ray. 2022. Natgen: generative pre-training by "naturalizing" source code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 18–30.

[5] CodeGeeX. 2023. CodeGeeX. https://models.aminer.cn/codegeex/blog/

[6] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, et al. 2023. CROSSCODEEVAL: A Diverse and Multilingual Benchmark for Cross-File Code Completion. *arXiv preprint arXiv:2310.11248* (2023).

[7] Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2022. Cocomic: Code completion by jointly modeling in-file and cross-file context. *arXiv preprint arXiv:2212.10007* (2022).

[8] Zishuo Ding, Yiming Tang, Xiaoyu Cheng, Heng Li, and Weiyi Shang. 2023. LoGenText-Plus: Improving Neural Machine Translation-based Logging Texts Generation with Syntactic Templates. *ACM Transactions on Software Engineering and Methodology* (2023).

[9] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, and Michael R Lyu. 2023. Constructing Effective In-Context Demonstration for Code Intelligence Tasks: An Empirical Study. *arXiv preprint arXiv:2304.07575* (2023).

[10] GitHub. 2023. GitHub Copilot: Your AI pair programmer. https://github.com/features/copilot

[11] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R Lyu. 2023. Do not give away my secrets: Uncovering the privacy issue of neural code completion tools. *arXiv preprint arXiv:2309.07639* (2023).

[12] JetBrains. 2023. idea. https://www.jetbrains.com/help/idea/code-inspection.html

[13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[14] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[15] Shuai Lu, Nan Duan, Hojae Han, Daya Guo, Seung-won Hwang, and Alexey Svyatkovskiy. 2022. Reacc: A retrieval-augmented code completion framework. *arXiv preprint arXiv:2203.07722* (2022).

[16] Microsoft. 2023. Pylance. https://marketplace.visualstudio.com/items?itemName=ms-python.vscode-pylance

[17] Microsoft. 2023. vscode. https://code.visualstudio.com

[18] NumPy. 2023. numpy. https://numpy.org

[19] OpenAI. 2021. CodeX. https://openai.com/blog/openai-codex

[20] OpenAI. 2023. ChatGPT. https://openai.com/blog/chatgpt/

[21] Yun Peng, Chaozheng Wang, Wenxuan Wang, Cuiyun Gao, and Michael R Lyu. 2023. Generative Type Inference for Python. *arXiv preprint arXiv:2307.09163* (2023).

[22] James F Power and Brian A Malloy. 2000. Symbol table construction and name lookup in ISO C++. In *Proceedings 37th International Conference on Technology of Object-Oriented Languages and Systems. TOOLS-Pacific 2000*. IEEE, 57–68.

[23] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).

[24] Disha Shrivastava, Denis Kocetkov, Harm de Vries, Dzmitry Bahdanau, and Torsten Scholak. 2023. RepoFusion: Training Code Models to Understand Your Repository. *arXiv preprint arXiv:2306.10998* (2023).

[25] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. *arXiv preprint arXiv:2309.00608* (2023).

[26] Chunqiu Steven Xia and Lingming Zhang. 2023. Conversational automated program repair. *arXiv preprint arXiv:2301.13246* (2023).

[27] Daoguang Zan, Bei Chen, Yongshun Gong, Junzhi Cao, Fengji Zhang, Bingchao Wu, Bei Guan, Yilong Yin, and Yongji Wang. 2023. Private-library-oriented code generation with large language models. *arXiv preprint arXiv:2307.15370* (2023).

[28] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570* (2023).

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 16:37:03 UTC from IEEE Xplore. Restriction